

Password Security Demo

Pierre Olivier

The University of Manchester, Department of Computer Science

`pierre.olivier@manchester.ac.uk`

Objective

- **Password hygiene/good practices** include among other things:
 - Passwords should be **long**, i.e., made of numerous characters/words **AND**
 - Passwords should be **complex**, i.e., made of various types of characters

Why such rules?

Objective

- **Password hygiene/good practices** include among other things:
 - Passwords should be **long**, i.e., made of numerous characters/words **AND**
 - Passwords should be **complex**, i.e., made of various types of characters

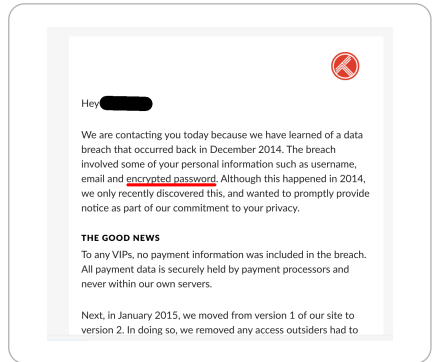
Why such rules?

Ethical Use Disclaimer

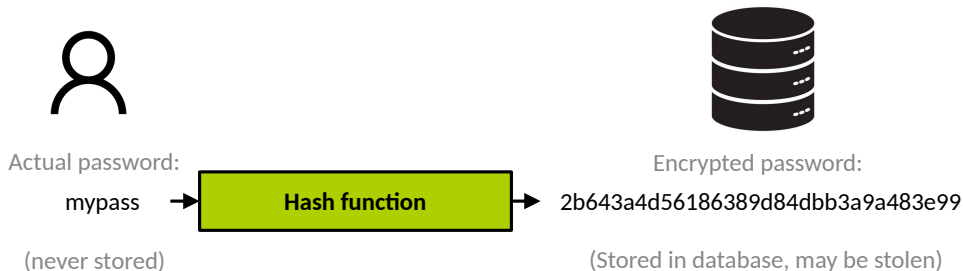
- This demo introduces techniques that are commonly associated with offensive security, their purpose in this context is purely educational
- You are expected to use the knowledge and skills from this demo responsibly and ethically
- Any use of these techniques outside of authorised, educational, or professional penetration testing contexts is strictly prohibited and may be illegal

Data Breaches

- Data breaches are very common
- One of the worst type of personal data that can be stolen are **passwords**

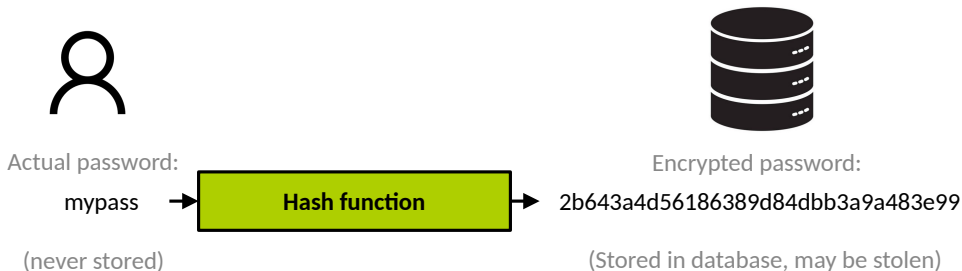


What's an Encrypted Password?



- Plain text passwords are transformed into **hashed (encrypted) passwords** stored into web services databases
 - Goal is to avoid storing plain text passwords

What's an Encrypted Password?



- Plain text passwords are transformed into **hashed (encrypted) passwords** stored into web services databases
 - Goal is to avoid storing plain text passwords
- Authentication process:
 1. Prompt user for plain text password attempt
 2. Hash the attempt
 3. Compare to the hash of the correct password in the database: if they match, authentication succeeds

Cracking Passwords



Steal
hashes

5d41402abc4b2a76b9719d911017c592
76a2173be6393254e72ffa4d6df1030a
5ebe2294ecd0e0f08eab7690d2a6ee69
a029d0df84eb5549c641e04a9ef389e5
84675f2baf7140037b8f5afe54eef841
098f6bcd4621d373cade4e832627b4f6
385aa5385e83efc5d83c7588ee5fb7e8
4f09daa9d95bcb166a302407a0e0babe
49f68a5c8493ec2c0bf489821c21fc3b
1a1dc91c907325c69271ddf0c944bc72



Crack
passwords

hello
passwd
secret
mypass
pierre
test
phrase
short
hi
pass



- Hackers commonly steal lists of **hashed passwords** from online services and applications
- Their next goal is then to **revert the hashes into plain text passwords**

Cracking Passwords (2)



- Directly reverting a hash back into the corresponding password is impossible
 - Hash functions are unidirectional transformations

Cracking Passwords (2)



- Directly reverting a hash back into the corresponding password is impossible
 - Hash functions are unidirectional transformations
- Common approach: attacker hashes themselves possible passwords and compare to stolen hashes
 - If they match: password recovered

Cracking Passwords (2)



- Directly reverting a hash back into the corresponding password is impossible
 - Hash functions are unidirectional transformations
 - Common approach: attacker hashes themselves possible passwords and compare to stolen hashes
 - If they match: password recovered
1. **Brute force attack:** try every possible combination of characters
 2. **Dictionary attack:** try commonly used passwords, e.g., dictionary words

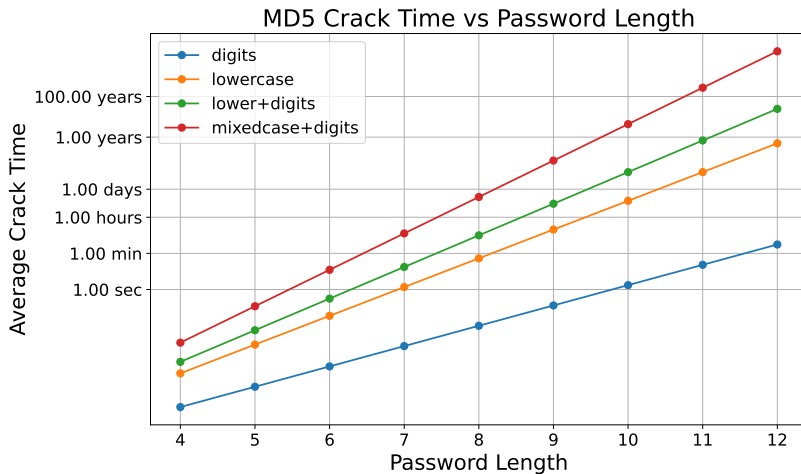
Cracking Passwords (2)



- Directly reverting a hash back into the corresponding password is impossible
 - Hash functions are unidirectional transformations
- Common approach: attacker hashes themselves possible passwords and compare to stolen hashes
 - If they match: password recovered
- 1. **Brute force attack:** try every possible combination of characters
- 2. **Dictionary attack:** try commonly used passwords, e.g., dictionary words
- Very efficient on weak passwords with the processing power of modern computers

Demo 1: Cracking a Simple Password

MD5 Cracking Speed



Demo 2: Cracking a List of Hashes

Good practices for strong password security **make it very hard/impossible to revert hashes**

- On the end user side:
 - **Choose long (12+ characters) passwords**
 - Choose passwords with a wide set of characters: digits, special characters, etc.
- On the application/website developer side:
 - Force user to choose long/varied passwords
 - Use strong hash methods, make it at least a bit computationally expensive to hash
 - Combine passwords with random data (salt) before hashing

Slides + Demo are Online

- You can try this demo (responsibly!) at home
- Instructions on how to reproduce it + PDF of the slides are available online:



<https://github.com/olivierpierre/scale-up-password-security-demo>